



INSTITUT INFORMATIQUE SUD AVEYRON

*Reclassement professionnel*

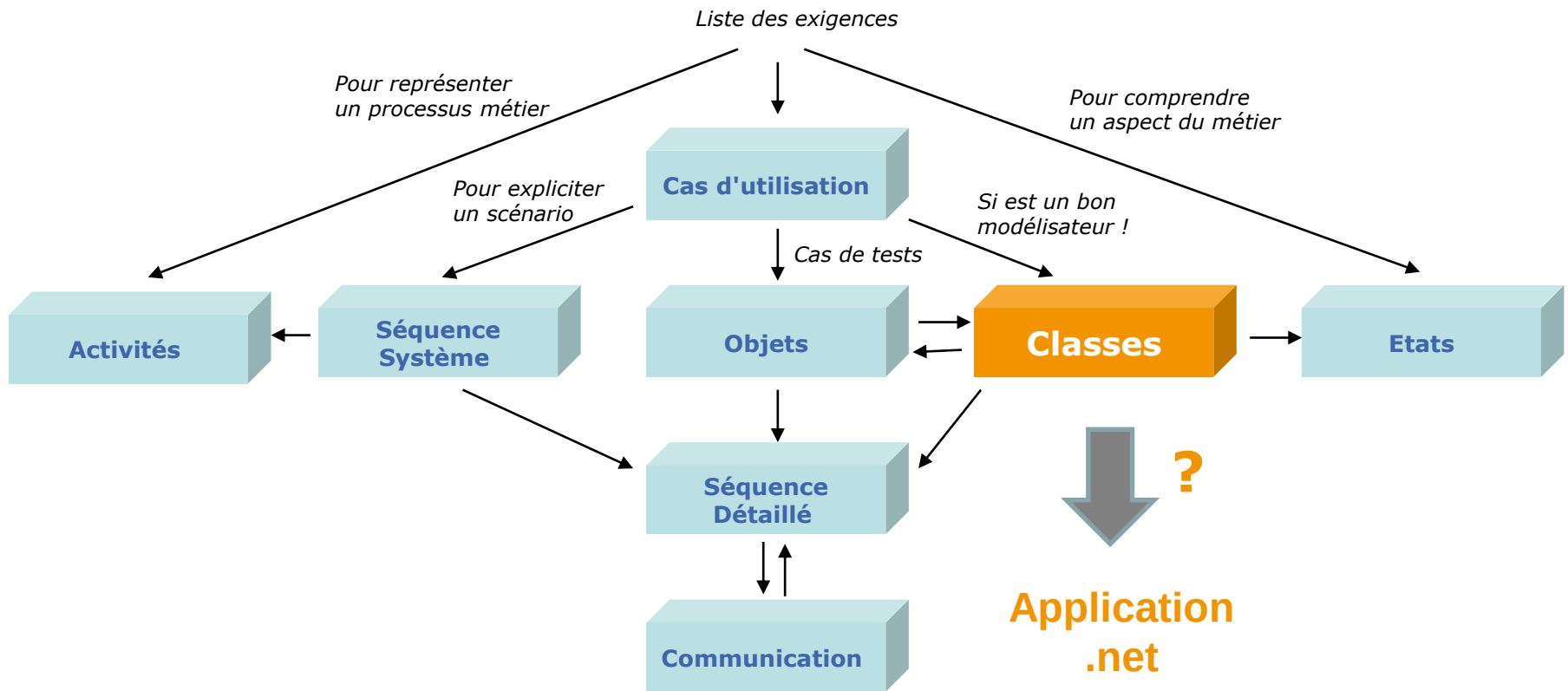
# Des diagrammes UML au code des classes

En C#

Comment passer des diagrammes de classes au code en C#

- ▶ Classes, attributs, opérations
- ▶ Généralisation/spécialisation
- ▶ Association, agrégation et composition
- ▶ Classe d'association
- ▶ Dépendance
- ▶ Interface et réalisation d'interfaces
- ▶ Quid des autres diagrammes ?

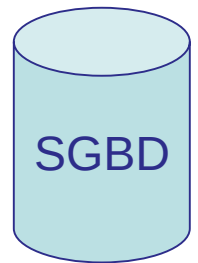
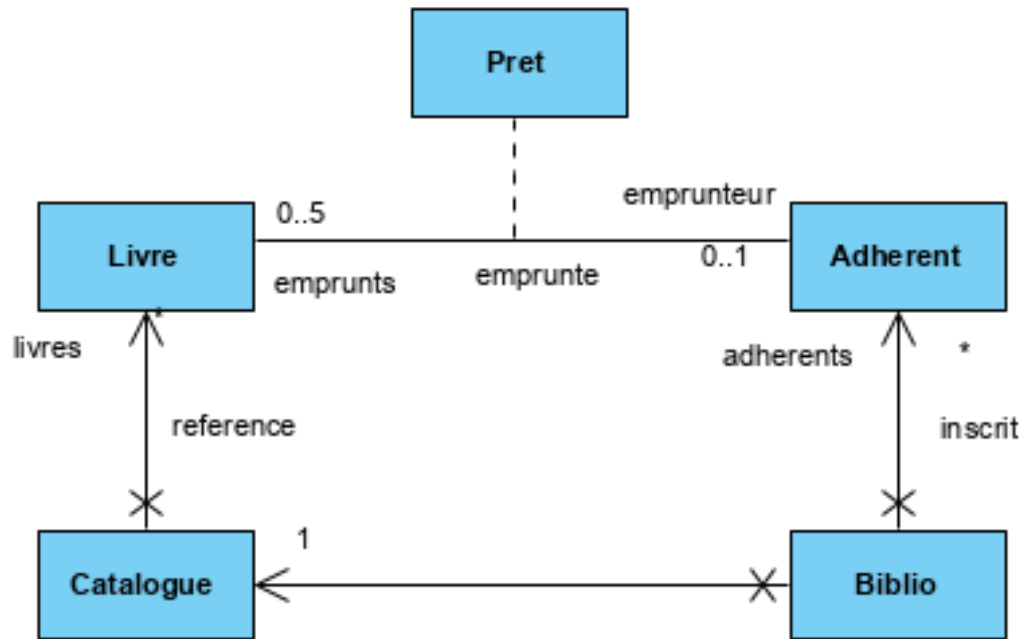
# Le problème



# Diagramme des classes



?

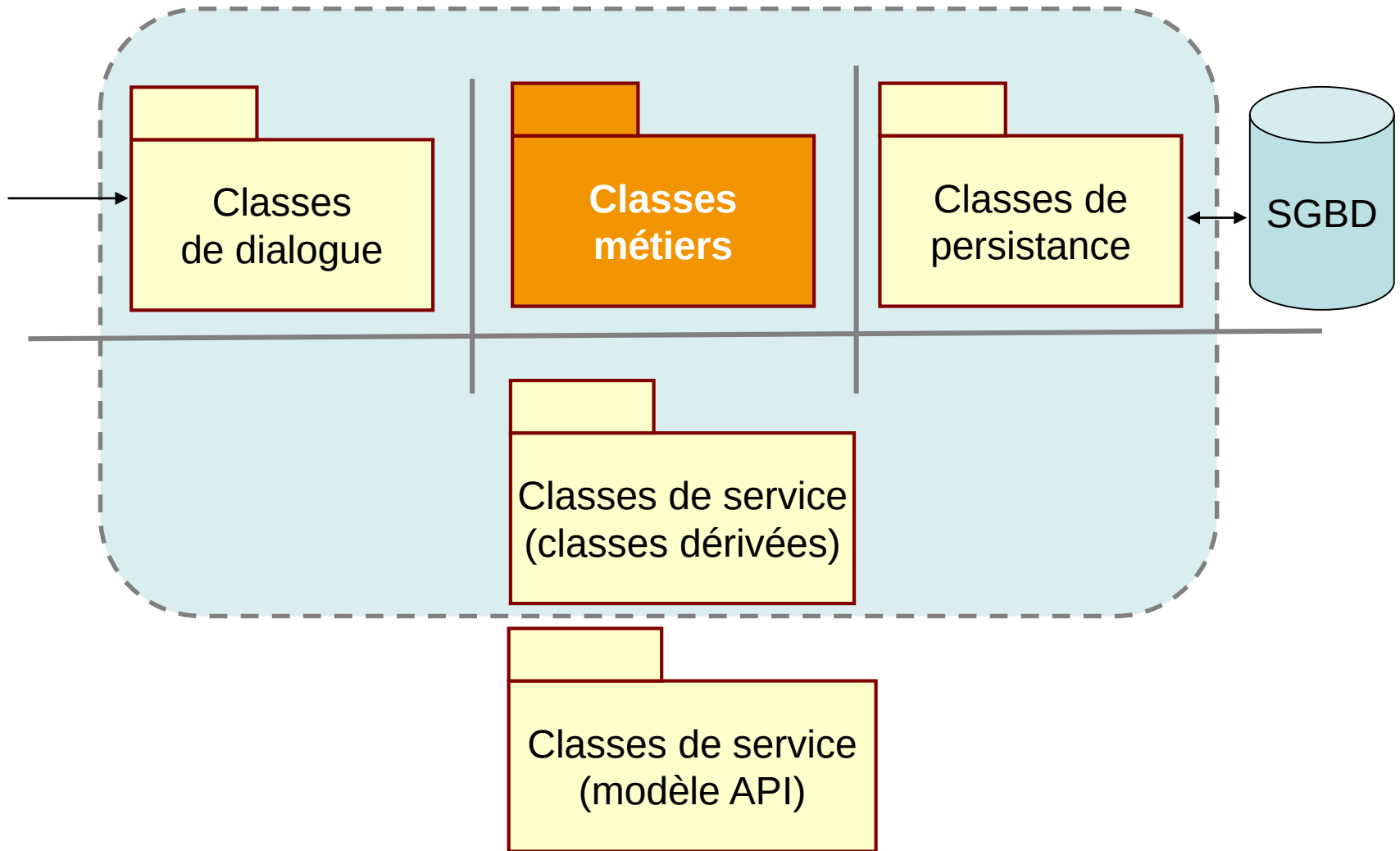


?

Classes métier

?

# Diagramme de classes étendu

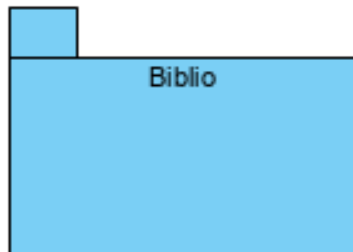


- Une classe UML = un fichier .cs



```
public class Adherent
{
    ...
}
```

- Les classes sont regroupées en namespaces (packages)

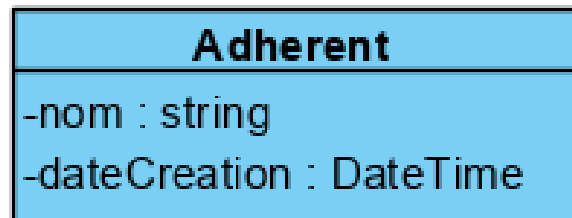


```
public namespace Biblio
{
    public class Adherent
    {
        ...
    }
}
```

# Attributs d'instance

## ► Les attributs deviennent des variables connues dans la toute la classe

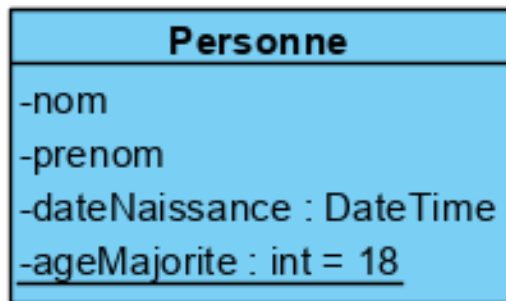
- Type
  - ✓ *Primitif* : int, string, double
  - ✓ *Classe fournie par la plateforme*
- Visibilité
  - *private*
  - # *protected*
  - + *public*



```
public class Adherent
{
    private string nom;
    private DateTime dateCreation;
}
```

# Attributs de classe

- Les attributs de classe deviennent des membres statiques



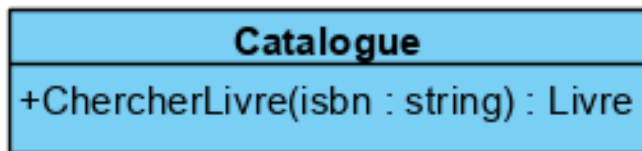
```
public class Personne
{
    private string nom;
    private string prenom;
    private Date    dateNaissance;

    private static int ageMajorite = 18;
    ...
}
```



# Opérations d'instance

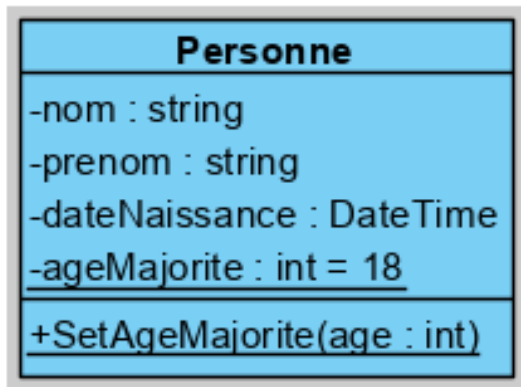
- Les opérations deviennent des méthodes : fonctions ou procédures
  - Visibilité : même conventions que les attributs
    - *private*
    - # *protected*
    - + *public*



```
public class Catalogue
{
    public Livre ChercherLivre (string isbn)
    {
        ...
    }
}
```

# Opérations de classe

- Les opérations de classe deviennent des méthodes statiques

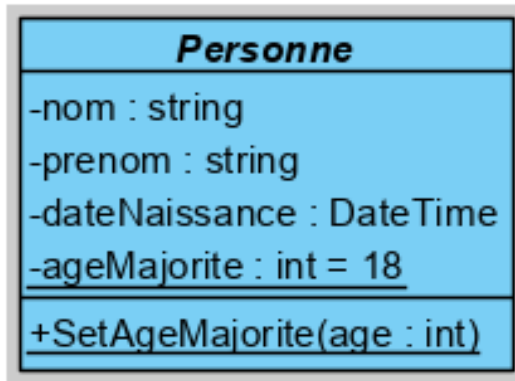


```
public abstract class Personne
{
    private string      nom;
    private string      prenom;
    private DateTime    dateNaissance;
    private static int  ageMajorite = 18;

    public static void SetAgeMajorite(int age)
    {
        ...
    }
}
```

# Classe abstraite

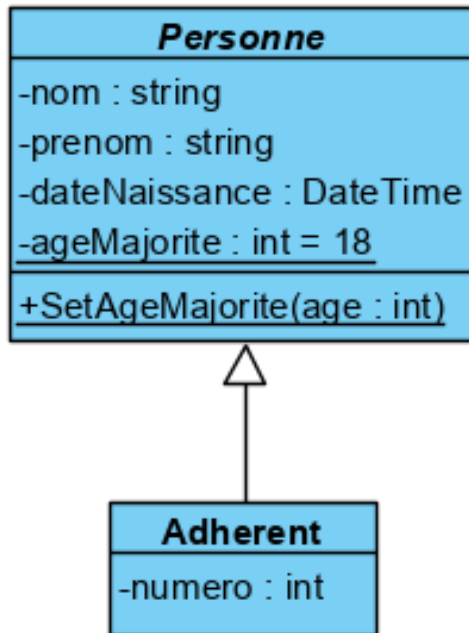
- ▶ Factorise un comportement
- ▶ Ne peut pas être instanciée



```
public abstract class Personne
{
    private string      nom;
    private string      prenom;
    private DateTime    dateNaissance;
    private static int  ageMajorite = 18;
    public int GetAge()
    { ...
    }
    public static void SetAgeMajorite(int age)
    { ...
    }
}
```

# Généralisation / spécialisation

- ▶ Une classe hérite au plus d'une classe de base
- ▶ Traduit « *est une sorte de* »



```
public class Adherent : Personne
{
    private int numero;
    ...
}
```

# Association

- Association **navigable** avec une **multiplicité 1**
  - Se traduit par une variable d'instance dont le type est une référence vers une instance d'une classe du modèle



```
public class A
{
    private B leB;
    ...
}
```

```
public class B
{
    ...
}
```

# Association

- Association **navigable de multiplicité \***
  - Se traduit par un attribut de type collection de références d'objets : **tableaux d'objets**, **ArrayList**, **List**, **Dictionary**



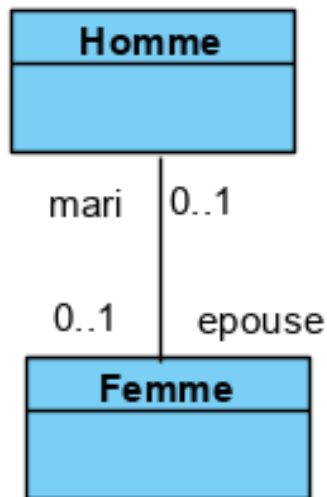
```
public class A
{
    private B[] lesB;
}
```

```
public class A
{
    private List<B> lesB = new List<B>();
}
```

# Association

## ► Association bidirectionnelle

- Se traduit par une paire de références  
→ une dans chaque classe impliquée
- Le nom des rôles aux extrémités servent à nommer les variables d'instance de type référence

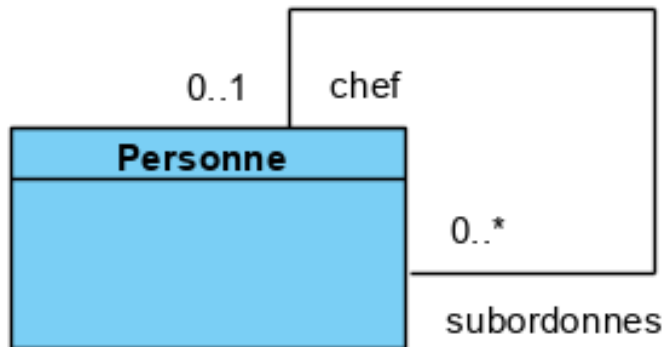


```
public class Homme
{
    private Femme epouse;
    ...
}
```

```
public class Femme
{
    private Homme mari;
    ...
}
```

# Association réflexive

- Se traduit par une référence sur un ou des objets de la même classe

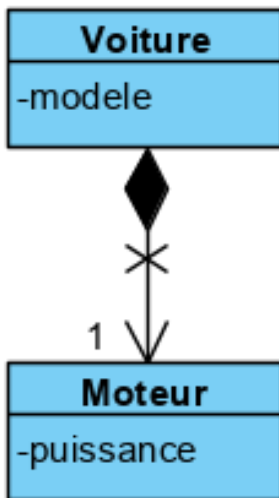


```
public class Personne
{
    private Personne chef;
    private Personne[] subordonnes;
    ...
}
```



# Agrégation & Composition

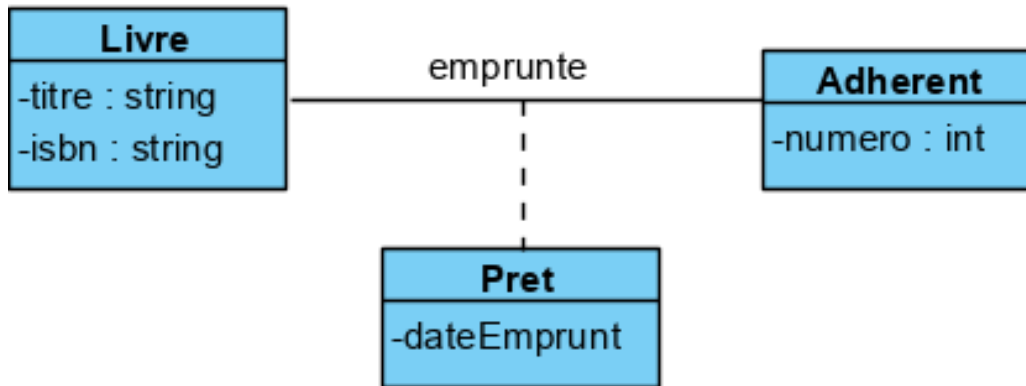
- ▶ L'agrégation est un cas particulier d'association
- ▶ Une composition est une agrégation plus forte :
  - Une partie ne peut appartenir qu'à un seul tout
  - Destruction du tout → destruction des parties



```
public class Voiture
{
    private string modele;
    private Moteur moteur;

    private class Moteur
    {
        private int puissance;
        ...
    }
}
```

# Classe d'association

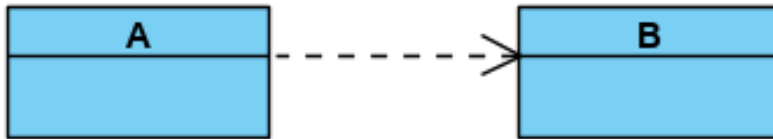


```
public class Pret
{
    private DateTime dateEmprunt;

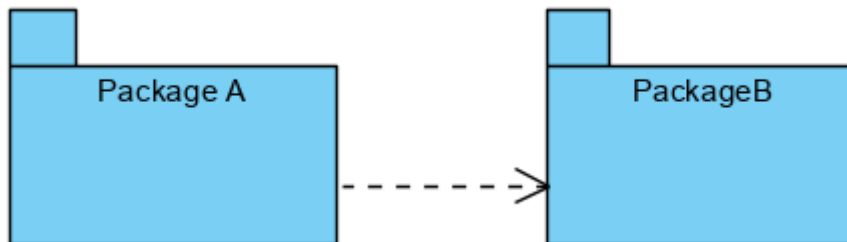
    private Livre livre;
    private Adherent emprunteur;
    ...
}
```

# Relation de dépendance

- ▶ Une classe A **dépend** d'une classe B
  - Si A a une méthode ayant comme paramètre une référence sur une instance de la classe B
  - Si A appelle une méthode de la classe B



- ▶ Dépendance entre packages
  - Se traduit par des **directives d'usage**



# Interface

- ▶ Définit un ensemble de prototypes de méthodes
- ▶ Décrit un comportement attendu par des classes

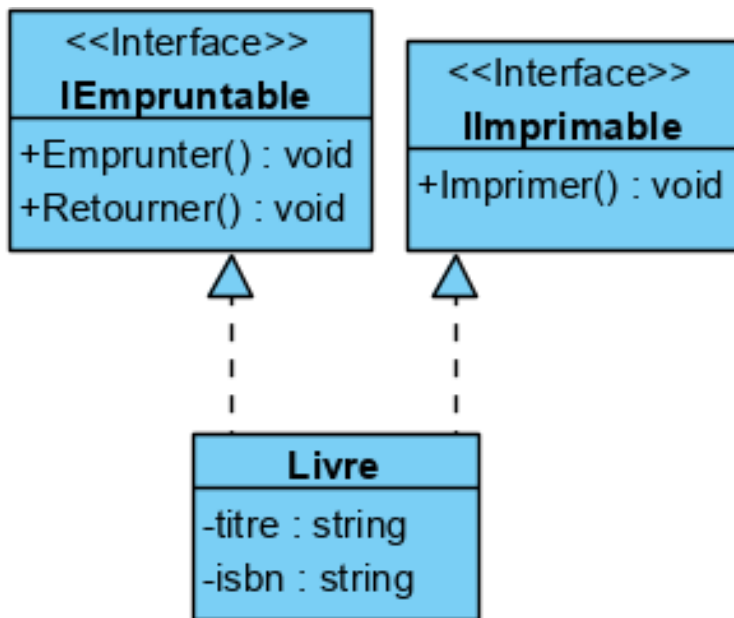


```
public interface IImprimable
{
    void Imprimer();
}
```



# Réalisation d'interface

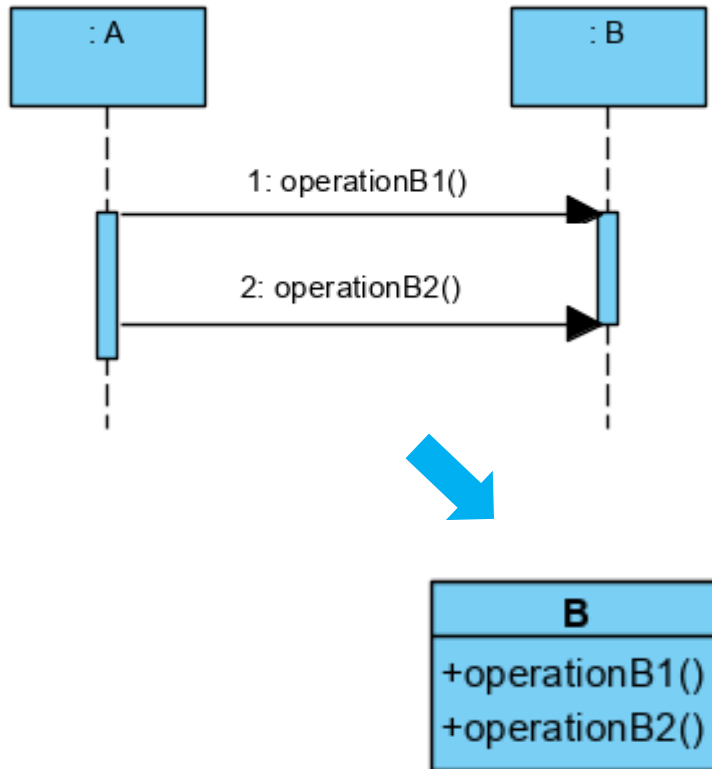
- Une classe peut implémenter plusieurs interfaces



```
public class Livre : IImprimable,
                    IEmpruntable
{
    private string titre;
    Private string auteur;
    private string isbn;
    ...
    public void Imprimer()
    {
        ...
    }
    public void Emprunter()
    {
        ...
    }
    public void Retourner()
    {
        ...
    }
}
```

# Interactions dynamiques

## ► Diagramme de séquence détaillé



```
public class A {
...
    leB.OperationB1 ();
... leB.OperationB2 ();
...
}

public class B {
...
    public void OperationB1 () {}
    public void OperationB2 () {}
...
}
```

# En savoir plus

---

 UML2 par la pratique – Pascal ROQUES (EYROLLES)